Polynomial Input Preconditioning for Zero-Shot Time Series Forecasting

Anonymous Author(s)

Abstract

Universal Sequence Preconditioning (USP) is a recent theoretical framework for online sequence prediction that preprocesses observations via a fixed polynomial convolution, yielding hidden-dimension-free sublinear regret for marginally stable linear dynamical systems. This paper investigates whether these techniques transfer empirically to zero-shot patch-based transformer time-series forecasters such as Moirai 2.0. We propose **polynomial input preconditioning**, where we concatenate the preconditioned signal as an auxiliary input channel while leaving the forecast target unchanged. By only adding 0.11% parameters, our method improves a Moirai 2.0 Small baseline by 2.9% geometric-mean MASE on GIFT-Eval and achieves similar gains on FEV-Bench. At a 100K-step budget, the method remains substantially better than the matched baseline, with a larger relative gap and lower cross-seed variance. Polynomial input preconditioning performs better with longer prediction horizon lengths, attaining a 5.3% improvement on long horizon prediction tasks. Capacity-matched zero and duplicate-channel controls show that the gain comes from the polynomial content rather than extra parameters. These results suggest that time-series foundation models can benefit substantially from theory-guided structure injected before tokenization, offering a lightweight alternative to improving model performance without scaling model size or attention complexity.

1 Introduction

Universal sequence preconditioning. In 2025, based on the theory of linear dynamical systems, Marsden and Hazan [24] proposed Universal Sequence Preconditioning (USP) for online learning. The technique generalizes differencing; in particular, they suggest preprocessing the inputs via a fixed convolution of polynomial coefficients,

$$x_t^{\text{pre}} \;=\; \sum_{k=0}^{d} c_k \, x_{t-k},$$

where c_k are fixed coefficients and d is the polynomial degree. They prove that this can result in hidden-dimension-free (up to logarithmic factors) sublinear regret with respect to the best LDS predictor.

Does this transfer to zero-shot time-series forecasting? We investigate how the USP framework can be applied to pretrained time series foundation models, and specifically to the Moirai 2.0 model. The setting of pretrained time series foundation models [3, 4, 8, 13, 21, 32] differs from USP in two important ways. First, the forecaster is trained offline on a large corpus and deployed for zero-shot prediction over a multi-step horizon, rather than making one-step predictions in an online loop where the true observation is revealed after each step. Second, the training data comes from many heterogeneous sources that may not be well-modelled as finite-dimensional LDSs.

Our approach. A direct way to use USP would be to train the forecaster to predict the preconditioned target sequence and then invert the preconditioning filter to recover forecasts in the original scale. For multi-step forecasting, this inversion may lead to error accumulation as each recovered future value depends on earlier recovered values, so forecast errors are fed back into later predictions. To address this we can leave the forecast target unchanged and concatenate the preconditioned signal as an auxiliary input channel alongside the raw target and the observation mask. Because the model

never operates in preconditioned space at inference, no inversion is needed and error accumulation is avoided.

A second motivation comes from the architecture of Moirai 2.0 itself. Patch-based transformers [9, 26] split the input into non-overlapping length-P windows and embed each patch independently, creating a cross-patch information bottleneck: before the first self-attention layer, each patch embedding encodes only P time steps, so multi-patch correlations must be discovered entirely through attention. A polynomial preconditioner that mixes information across the same multi-patch scales before patching can inject cross-patch structure into every embedding at negligible parameter cost.

Contributions. (1) We introduce polynomial input preconditioning, a simple modification adding a fixed Chebyshev FIR-filtered channel to patch-based time series transformers. (2) Through 5-seed experiments on 97 GIFT-Eval configurations, we show a 2.9% improvement ($p < 10^{-15}$) and control for confounds: a duplicate channel hurts, a zero channel is neutral. At 100K steps, the benefit grows to 3.9%. (3) Ablations identify degree 4, patch-aligned stride, Chebyshev basis, and 10% hint dropout as optimal. (4) Domain, frequency, and horizon breakdowns show the benefit is strongest where the filter's lookback scale aligns with structured temporal patterns. (5) The result is confirmed on the independent FEV-Bench benchmark (100 tasks, -2.2% MASE, 78/100 wins).

2 Related Work

2.1 Time Series Foundation Models

Recent work trains large models on diverse time series corpora for zero-shot forecasting. Decoder-only approaches include TimesFM [8] and Lag-Llama [29]. Tokenization-based methods include Chronos [3], which trains a T5 model on discretized values. Masked encoder approaches include Moirai [32] with any-variate attention and frequency-adaptive patching, Moirai 2.0 [21] which introduces a decoder-only architecture, MOMENT [13], and TTM [10]. TimeGPT [12] and Timer [23] use generative pre-training, while UniTS [11] unifies multiple tasks. Several surveys [20, 27] and critical evaluations [35] discuss the landscape. We build on Moirai 2.0 Small (11.4M parameters), which uses the LOTSA corpus [32] containing 27B observations across 9 domains.

2.2 Patching and Cross-Patch Information

PatchTST [26] introduced channel-independent patching for time series, splitting the input into non-overlapping windows of size P and projecting each independently before transformer processing. This approach was adopted by Moirai [32] and TimesFM [8]. The patch-level tokenization introduces an information bottleneck: each initial embedding is blind to temporal patterns spanning multiple patches. PatchTST partially addresses this through optional patch overlap, and Han et al. [16] analyze the capacity–robustness tradeoff of channel-independent processing. Alternatives that address cross-variate or cross-patch interactions include iTransformer [22] (variate-level attention), Crossformer [34] (cross-dimension attention), and multi-patch prediction [6]. Our approach is orthogonal: rather than modifying the attention mechanism or patching strategy, we inject cross-patch information at the input level through a fixed polynomial filter, requiring no changes to the transformer stack.

2.3 Spectral Methods in Sequence Modelling

Spectral approaches to sequence modelling have a rich history. Hazan et al. [17] introduced spectral filtering for learning linear dynamical systems, extended by Hazan et al. [18] to general LDS. The recently proposed spectral state space models [1] and Flash STU [7] leverage polynomial structure for efficient sequence processing. In time series specifically, FEDformer [37] uses frequency-enhanced decomposition, FiLM [36] applies Legendre polynomial projections for long-term memory, and FreTS [33] operates entirely in the frequency domain. Structured state space models [15] and their selective variant Mamba [14] also use polynomial-like parameterizations. Our work bridges the gap between the theoretical USP framework [24] and practical patch-based forecasters, showing that a fixed polynomial FIR filter applied before tokenization provides a consistent benefit.

2.4 Input Preprocessing for Time Series

Input transformations are a well-established technique for improving time series models. Reversible instance normalization (RevIN) [19] applies learnable affine normalization to handle distribution shift. DAIN [28] uses adaptive input normalization. Classical differencing [5] removes trends and non-stationarity. WaveNet [31] uses dilated causal convolutions to capture multi-scale temporal patterns. Our polynomial preconditioning can be seen as a generalization of first-order differencing: a degree-d Chebyshev polynomial at stride s provides a principled multi-scale lookback that captures structure at $2s, 4s, \ldots, ds$ lags, with coefficients derived from approximation theory rather than learned.

3 Method

3.1 Background: Patch-Based Time Series Transformers

Consider a univariate time series $x = (x_1, \ldots, x_T)$ with observation mask m. Following Moirai [21, 32], the model applies instance normalization, then partitions the series into non-overlapping patches of size P: $\mathbf{p}_i = (x_{(i-1)P+1}, \ldots, x_{iP})$. Each patch and its mask form a $2P$-dimensional input projected to d_{model} dimensions via a residual block:

$$\mathbf{e}_i = \text{ResBlock}_{2P \to d}([\mathbf{p}_i; \mathbf{m}_i]). \tag{1}$$

Moirai 2.0 Small uses a 6-layer decoder-only transformer with $d_{\text{model}} = 384$, $d_{\text{ff}} = 1024$, 6 attention heads, causal multi-head self-attention with rotary position embeddings (RoPE), SwiGLU feed-forward blocks, and RMSNorm. The output projection is a ResBlock mapping $\mathbb{R}^d \to \mathbb{R}^{K \times |\mathcal{Q}| \times P}$ where K=4 is the multi-token horizon and $|\mathcal{Q}|$=9 the number of quantile levels.

The key observation is that before the first self-attention layer, each patch embedding encodes only P consecutive time steps. Temporal patterns spanning multiple patches—daily cycles in hourly data (24 steps = 1.5 patches at $P = 16$), weekly seasonality in 15-minute data (672 steps = 42 patches)—are invisible to individual embeddings and must be recovered entirely through attention.

3.2 Preconditioning Polynomial

Given a normalized input series x, let $\tilde{T}_d(z) = z^d + c_1 z^{d-1} + \cdots + c_d$ be the degree-d monic Chebyshev polynomial in descending powers. Following the implementation convention, the leading monic term is the current value and the remaining coefficients are applied at stride-s lags:

$$r_t = x_t^{\text{pre}} - x_t = \sum_{k=1}^{d} c_k \, x_{t-ks}. \tag{2}$$

where s is a stride parameter set equal to the patch size P. Our default is the monic Chebyshev polynomial of the first kind, $\tilde{T}_d$, with $d = 4$. For $P = 16$ the coefficients are $(c_1, \ldots, c_4) = (0, -1, 0, \frac{1}{8})$, so $r_t = -x_{t-32} + \frac{1}{8} x_{t-64}$.

The choice of Chebyshev polynomials is motivated by their optimality in minimax polynomial approximation [25] and their well-conditioned spectral properties in the context of spectral filtering for LDS prediction [17, 24].

3.3 Preconditioning Channel

The residual r is partitioned into patches $\mathbf{r}_i$ aligned with the data patches $\mathbf{p}_i$. In Moirai 2.0, each patch passes through a per-patch input projection (a two-layer residual MLP with SiLU activation) that maps the concatenation of the data patch and an observation mask to an embedding vector. We widen this projection to accept a third input, the preconditioning patch:

$$\mathbf{e}_i = \text{ResBlock}_{3P \to d}([\mathbf{p}_i; \mathbf{m}_i; \mathbf{r}_i]). \tag{3}$$

The input projection ResBlock consists of:

$$\mathbf{z} = \text{SiLU}(W_1 \mathbf{x} + b_1), \qquad W_1 \in \mathbb{R}^{d \times 3P}, \tag{4}$$

$$\mathbf{o} = W_2 \mathbf{z} + b_2 + W_3 \mathbf{x} + b_3, \qquad W_2 \in \mathbb{R}^{d \times d}, \; W_3 \in \mathbb{R}^{d \times 3P}. \tag{5}$$

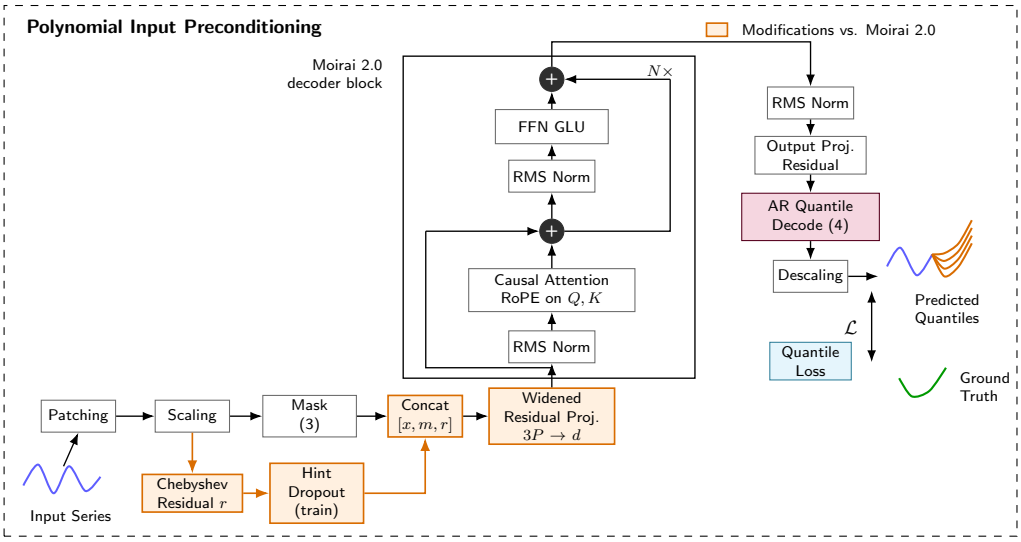

Figure 1: Polynomial input preconditioning in Moirai 2.0. Orange boxes mark components changed relative to the standard architecture: the Chebyshev residual channel, optional training-time hint dropout, concatenation of $[x, m, r]$, and the widened residual input projection from $2P$ to $3P$ dimensions. The decoder block and autoregressive quantile output head are unchanged. Total overhead: 12K parameters (0.11%).

Only W_1 and W_3 are wider than the baseline. W_2 and all downstream layers are identical. The input dimension grows from $2P$ to $3P$, adding 12K parameters (0.11% of Moirai 2.0 Small). The transformer decoder is unchanged.

3.4 Channel Dropout

During training with channel dropout probability p, each patch's preconditioning channel is independently zeroed with probability p, preventing overdependence on the preconditioning signal. At inference, the full preconditioning channel is always provided. We refer to the default configuration ($d = 4$ Chebyshev, $s = 16$, no dropout) as `d4`, and to the same architecture trained with 10% per-patch channel dropout as `d4_dropout`.

The z-score normalization that precedes the residual computation uses only observed context values; prediction-window positions are excluded, so no future information leaks into r_t. Unobserved time steps are masked to zero.

4 Experiments

4.1 Setup

We train Moirai 2.0 Small [21] (11.4M params, 6-layer transformer, P=16) from scratch on the public LOTSA corpus [32] for 10K steps (batch 256, AdamW, cosine LR 10^{-3}, 1K warmup, bf16). The official Moirai 2.0 model was trained for 100K steps on 295B observations, while our accessible LOTSA setup contains 27B observations and is run for either 10K or 100K steps. All comparisons in this paper use identical data, optimizer, loss, precision, and seeds within the compared conditions. The complete training protocol is detailed in Appendix A.

We then train models `d4` and `d4_dropout` with the architectural modifications described above. For `d4_dropout`, the preconditioning channel is independently zeroed with probability 0.1 per patch during training and always provided at inference. All trained models share identical data, optimiser, seeds, and compute, so the effects we report are controlled relative improvements.

4.2 Evaluation

At inference, the model predicts multiple future patches per step. For longer horizons, we proceed with autoregressive decoding, where predicted patches are appended to the context and the decoder is

Table 1: Main results on GIFT-Eval (5 seeds). MASE is averaged per configuration across seeds, then aggregated via geometric mean. Wins: configurations where the method's mean MASE $<$ Baseline's (out of 97); p-value is calculated from a two-sided paired sign test.

Method	MASE	$\Delta\%$	Wins	p
`d4_dropout` (Ours)	**0.837**	-2.9%	**72/97**	$< 10^{-5}$
`d4` (Ours)	0.838	-2.8%	72/97	$< 10^{-5}$
Zero ctrl	0.858	-0.5%	54/97	0.31
Baseline	0.862	—	—	—
Duplicate ctrl	0.865	$+0.4\%$	39/97	0.07

re-run with the preconditioning residual recomputed over the extended context.

Evaluation uses GIFT-Eval [2] (97 dataset$\times$horizon configs) at context length 4,000 and the independent FEV-Bench [30] (100 tasks). GIFT-Eval is a large-scale zero-shot benchmark for general and foundation time-series forecasters, spanning 23 datasets, over 144,000 time series, seven domains, ten sampling frequencies, multivariate inputs, and short- to long-horizon tasks. FEV-Bench is a realistic forecasting benchmark for reproducible and statistically sound model comparison, comprising 100 tasks from 96 datasets across seven real-world domains, including covariate-rich univariate and multivariate settings. We report normalized MASE following the GIFT-Eval leaderboard convention: each configuration's MASE is divided by the seasonal naive baseline's MASE, then aggregated via geometric mean. We run 5 seeds (0, 1, 2, 7, 42) and report cross-seed mean and standard deviation.

4.3 Conditions

We compare four conditions, all sharing the same training data, optimizer, and seeds. **Baseline** is unmodified Moirai 2.0 (no preconditioning channel). **`d4`** and **`d4_dropout`** add the polynomial preconditioning channel of §3 (without and with 10% per-patch channel dropout). **Zero** and **Duplicate** are capacity-matched controls: they share `d4_dropout`'s widened input projection and 12K extra parameters, but feed the third channel either all zeros (Zero) or a copy of the data channel (Duplicate). This helps us determine whether changes in performance are due to the polynomial content or just from the extra capacity.

4.4 Main Results

Both `d4` and `d4_dropout` improve over Baseline on every seed (Table 1). The average improvements across seeds are 2.8% and 2.9% respectively. Their average MASE values are within 0.1% of each other, so the bulk of the gain comes from the polynomial content rather than the dropout. Zero and Duplicate share `d4_dropout`'s architecture and 12K-parameter overhead. Duplicate hurts ($39/97$ wins) and Zero has no significant effect ($54/97$, $p = 0.31$). This demonstrates the importance of the polynomial content of the filter rather than just the additional capacity.

4.5 Per-Seed Consistency

Table 2 shows the per-seed results. `d4_dropout` wins on every seed, with effect sizes ranging from 0.44% (seed 1) to 4.99% (seed 0). Averaging each configuration's MASE across all 5 seeds, `d4_dropout` wins 72/97 configurations ($p_{\text{sign}} = 9.6 \times 10^{-7}$; $p_{\text{Wilcoxon}} = 9.0 \times 10^{-9}$). Both `d4` and `d4_dropout` achieve the lowest MASE on every seed compared to both capacity controls.

4.6 FEV-Bench Validation

On the independent FEV-Bench (Table 3, 5 seeds each), `d4_dropout` wins on $78/100$ tasks with $\Delta = -2.2\%$ MASE ($p < 10^{-7}$), confirming the benefit is not benchmark-specific.

5 Ablations

We perform comprehensive ablations over the polynomial degree, stride, basis type, dropout rate, and learned vs. fixed coefficients. The degree sweep uses 5 seeds; all other ablations use seed 0 with 10% channel dropout unless otherwise noted.

Table 2: `d4` and `d4_dropout` vs. Baseline: per-seed pairwise comparison (10K steps, $\eta = 10^{-3}$).

Seed	BL	d4	d4_dropout	d4 Δ%	d4_drop Δ%
0	0.8666	0.8327	0.8234	−3.91%	−4.99%
1	0.8503	0.8435	0.8465	−0.80%	−0.44%
2	0.8630	0.8283	0.8391	−4.02%	−2.77%
7	0.8676	0.8422	0.8320	−2.93%	−4.11%
42	0.8614	0.8425	0.8430	−2.19%	−2.13%
Mean	**0.8617**	**0.8379**	**0.8368**	−2.77%	−2.89%

Table 3: FEV-Bench (5 seeds each). SQL (scaled quantile loss) is FEV-Bench's primary metric. MASE values are unnormalized (unlike the normalized MASE in Table 1). Wins: per-task mean MASE of `d4_dropout` < Baseline (out of 100); p from paired sign test.

Method	MASE	SQL	Wins	p
d4_dropout (Ours)	**1.254**	**1.024**	**78/100**	$< 10^{-7}$
Baseline	1.282	1.045	—	—
Δ%	−2.2%	−2.0%		

5.1 Polynomial Degree

We sweep over the Chebyshev polynomial degree d from $d = 2$ to $d = 7$, over 5 seeds each (Figure 2, Table 4). Every tested degree improves over the baseline, and $d = 4$ gives the lowest mean MASE. The curve is shallow enough that we use $d = 4$ as a strong default rather than treating the degree as a sharply tuned hyperparameter.

5.2 Stride

Stride $s = 16$ (equal to patch size P) is optimal (Table 5): the nonzero coefficients align exactly with patch boundaries, providing maximally informative cross-patch lookback. When stride equals patch size, filter taps look back to the same position in neighbouring patches, so the hint for patch i contains values from patches $i-2$ and $i-4$.

5.3 Basis

Chebyshev is best in the seed-0 sweep (Table 6), while EMA is also competitive (-3.89%) and finite differences are weakest (-1.12%, not significant vs baseline). Chebyshev beats Legendre d=4 head-to-head on 67/97 configurations ($p < 0.001$). The ranking is consistent with the filter's spectral properties: Chebyshev provides minimax-optimal polynomial approximation, while Legendre and EMA offer good but suboptimal conditioning.

5.4 Channel Dropout

The dropout sweep (Table 7) shows an inverted-U pattern: 10% optimal, both lower and higher rates worse. Without dropout, the model becomes overly hint-dependent: replacing hints with zeros at inference degrades MASE by 35.9%. With 10% dropout, degradation is only 9.3% (Appendix K).

5.5 Fixed vs. Learned Coefficients

We also consider directly learning the preconditioning coefficients (Table 8). When initialized from the Chebyshev values, the learned coefficients stay close to the initialization across all seeds ($c_2 \approx -1.03$) but consistently drift at other lags. When initialized from zero, the coefficients converge to a smoothly decaying pattern whose sign varies by seed but whose magnitude is consistent ($|c_1| \approx 0.31$, $|c_2| \approx 0.18$). Both learned variants beat Baseline, but fixed Chebyshev still performs best, suggesting a genuine inductive bias. Full per-seed learned coefficients are reported in Appendix K.

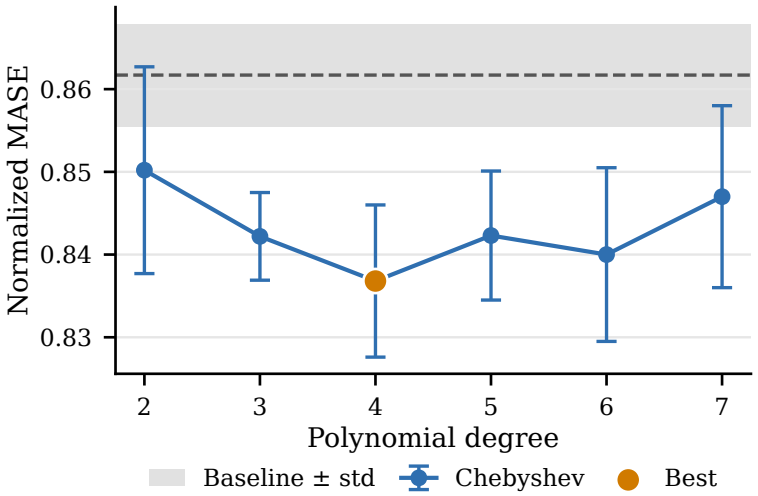

Figure 2: Chebyshev polynomial degree sweep (10K steps, stride s=16, 10% channel dropout). Filled blue circles are 5-seed means with ± 1 std error bars. The grey band is the baseline 5-seed mean ± 1 std.

Table 4: Polynomial degree sweep (5 seeds, 10K steps, Chebyshev, $s = 16$, 10% dropout). Residual lags count the nonzero entries of the hint coefficients $(c_1, \ldots, c_d)$ after the identity term has been stripped.

Degree	**Residual lags**	**MASE (mean $\pm$ std, n=5)**	**Δ% vs BL**
2	1 (at $2P$)	0.8502 ± 0.0125	-1.33%
3	1 (at $2P$)	0.8422 ± 0.0053	-2.27%
4	2 (at $2P$, $4P$)	$\mathbf{0.8368 \pm 0.0092}$	$\mathbf{-2.89\%}$
5	2 (at $2P$, $4P$)	0.8423 ± 0.0078	-2.25%
6	3 (at $2P$, $4P$, $6P$)	0.8400 ± 0.0105	-2.52%
7	3 (at $2P$, $4P$, $6P$)	0.8470 ± 0.0110	-1.71%
Baseline	—	0.8617 ± 0.0062	—

6 Extended Training

At 100K optimizer steps (Table 9, 5 seeds) the relative MASE reduction increases to -3.9% while both capacity controls now do worse than baseline (Duplicate $+0.9\%$, Zero $+1.7\%$). `d4_dropout` also shows lower cross-seed variance: raw standard deviation is 0.0079 versus 0.0189 for Baseline, equivalent to 0.9% versus 2.2% of mean MASE.

The per-seed 100K results (Table 10) reveal that the baseline shows higher variance across seeds at 100K (seed 0 reaches 0.9092 due to late-training instability), while `d4_dropout` remains stable (range 0.83–0.86). This suggests the polynomial preconditioning channel provides an inductive bias that compounds positively over longer training, while uninformative extra channels introduce optimization difficulties.

6.1 Training Dynamics

The training-loss curves in Figure 3 separate early and remain ordered throughout the run, with the gap widening after roughly 20K steps. This is consistent with the 100K evaluation results. The preconditioning channel is not merely a short-horizon optimization aid, but continues to improve the learned predictor as training proceeds.

Importantly, the training loss gap is consistently amplified into a larger eval MASE improvement: -1.0% train loss $\rightarrow -2.9\%$ eval MASE at 10K ($2.9\times$ amplification), and -2.4% train loss $\rightarrow -3.9\%$ eval MASE at 100K ($1.6\times$). Zero and Duplicate controls show the reverse: training loss comparable

Table 5: Stride ablation (seed 0, 10K steps, Chebyshev $d = 4$, 10% dropout).

Stride	MASE	Δ% vs BL	Wins/97
16 ($= P$)	**0.8234**	$\mathbf{-4.99\%}$	**66**
4	0.8406	-3.00%	57
8	0.8518	-1.71%	62
32	0.8484	-2.10%	63

Table 6: Basis ablation (seed 0, 10K steps, $d = 4$, $s = 16$, 10% dropout).

Basis	MASE	Δ%	Wins vs BL	Wins vs Cheby
Chebyshev	**0.8234**	$\mathbf{-4.99\%}$	66/97	—
EMA	0.8329	-3.89%	65/97	41/97
Legendre $d = 4$	0.8400	-3.07%	62/97	30/97
Legendre $d = 6$	0.8397	-3.10%	64/97	50/97
Finite Diff	0.8569	-1.12%	54/97	29/97
Duplicate	0.8583	-0.96%	42/97	27/97

to baseline but worse eval MASE at 100K, confirming that the preconditioning channel improves generalization rather than optimization speed. Full training loss tables are provided in Appendix B.

6.2 Horizon Stratification

On short horizons (Prediction Length (PL) ≤ 60) `d4` and `d4_dropout` are within 0.4% of each other (Table 11). On medium and long horizons (PL ≥ 480) `d4_dropout` beats `d4` by 0.7–0.85% relative MASE. The performance increase over baseline is greatest for the long horizon prediction tasks, where we attain a 5.3% MASE reduction.

This pattern is consistent with the training–inference mismatch created by autoregressive decoding. At longer horizons, the preconditioning channel increasingly depends on model-generated history rather than observed context; dropout exposes the model to missing preconditioning patches during training, making `d4_dropout` more robust when those signals become partially synthetic at inference.

7 Analysis

7.1 Domain and Frequency Analysis

The benefit is heterogeneous across domains. The strongest improvements appear in transport ($+6.9\%$, $p = 4 \times 10^{-8}$), energy ($+4.0\%$, $p = 2 \times 10^{-4}$), and demographics ($+9.8\%$, $p = 0.004$), while weather ($+0.3\%$, $p = 0.68$) and healthcare ($+0.2\%$, $p = 0.62$) are neutral. By frequency, the benefit is strongest at sub-hourly ($+3.4\%$) and hourly ($+3.8\%$) frequencies. This is consistent with the mechanism: at stride s=16 the filter looks back 32 and 64 steps—for 15-minute data, this captures intra-day patterns driving transport and energy demand, while for daily weather data the lookback spans 32–64 days, too long for relevant patterns. The benefit also increases with prediction horizon: $+1.4\%$ for short horizons to $+5.3\%$ for long horizons. Detailed domain and frequency tables are provided in Appendix E.

7.2 Mechanism: Cross-Patch Information Leakage

The preconditioning channel provides each patch embedding with information about *neighbouring patches* at the input level. For d=4, s=16: $r_t = -x_{t-32} + \frac{1}{8}\, x_{t-64}$, i.e., the preconditioning residual for patch i contains values from patches $i-2$ and $i-4$. This is conceptually related to overlapping patches in PatchTST [26], but more efficient: instead of increasing sequence length, we inject cross-patch information through a separate channel with zero overhead in the transformer encoder. The optimality of stride $s = P$ supports this interpretation—when filter taps align with patch boundaries, they provide maximally informative cross-patch signal.

Table 7: Channel dropout ablation (seed 0, 10K steps, Chebyshev $d = 4$, $s = 16$).

Dropout	MASE	Δ% vs BL	Wins/97
0% (d4)[†]	0.8378	-3.5%	—
2%	0.8388	-3.20%	63
3%	0.8497	-1.95%	55
10%	**0.8234**	$\mathbf{-4.99\%}$	**66**
20%	0.8422	-2.81%	67
30%	0.8559	-1.23%	61

[†] `d4` MASE is the 5-seed mean. `d4` and `d4_dropout` are essentially tied; dropout helps marginally.

Table 8: Fixed vs learned coefficients (5 seeds, 10K steps).

Setting	MASE	Std	Δ%
Fixed Chebyshev	**0.837**	0.008	$\mathbf{-2.9\%}$
Learned, zero init	0.839	0.006	-2.7%
Learned, Cheb. init	0.844	0.007	-2.1%

7.3 Learning Rate Interaction

`d4_dropout`'s MASE is nearly identical at $\eta = 10^{-3}$ (0.8368) and 2×10^{-3} (0.8361), a 0.09% difference, while the baseline varies by 2.27%. This suggests two benefits: (1) *optimization stability*—the polynomial structure acts as an implicit regularizer smoothing the loss landscape; (2) *genuine inductive bias*—at matched optimal LR, the polynomial content still provides 0.7% improvement ($p < 10^{-3}$). Across all 5 seeds, the training loss gap is consistently small relative to the eval gap. The matched learning rate comparison is detailed in Appendix D.

7.4 Inference Ablation

Training `d4_dropout` normally and evaluating with different inputs in the preconditioning channel confirms the model has learned specific polynomial structure:

Inference input	MASE	vs. Normal
Normal (Chebyshev)	0.8234	—
Baseline (no channel)	0.8666	$+5.2\%$
Zeros	0.9003	$+9.3\%$
Random noise	0.9427	$+14.5\%$
Duplicate (copy of input)	1.0699	$+29.9\%$

The ordering Normal $\gg$ Baseline $>$ Zero $>$ Random $>$ Duplicate confirms the model does not simply use the extra channel for generic capacity.

8 Limitations

We identify several limitations of our approach:

1. **Optimization stability vs. inductive bias.** The improvement at default LR (-2.9%) is larger than at matched optimal LR (-0.7%). This means a substantial portion of the benefit comes from optimization stability (smoothing the loss landscape) rather than a pure inductive bias effect, though the residual improvement at matched LR remains statistically significant ($p < 10^{-3}$).

2. **Domain-dependent benefit.** Weather and healthcare domains see no significant improvement, likely because the filter's lookback scale (32–64 time steps at stride 16) does not align with the relevant temporal patterns in these domains. Adapting the stride or degree per-domain could address this but would sacrifice the simplicity of a fixed, universal configuration.

Table 9: 100K training budget, 5 seeds. Normalized geomean MASE and raw cross-seed standard deviation are reported.

Method	MASE	Δ% vs BL	Std
d4_dropout (Ours)	**0.8443**	−3.88%	**0.0079**
Baseline	0.8784	—	0.0189
Duplicate (cap. ctrl)	0.8860	+0.86%	0.0083
Zero (cap. ctrl)	0.8937	+1.73%	0.0353

Table 10: Full 100K training budget with capacity controls (5 seeds). d4_dropout vs BL: −3.9%; d4_dropout vs Dup: −4.7%; d4_dropout vs Zero: −5.5%.

	s0	s1	s2	s7	s42	Mean
Baseline	0.9092	0.8920	0.8633	0.8641	0.8636	0.8784
d4_dropout	**0.8349**	**0.8402**	**0.8586**	**0.8450**	**0.8430**	**0.8443**
Duplicate	0.8887	0.8740	0.8974	0.8794	0.8903	0.8860
Zero	0.8917	0.8585	0.8750	0.8825	0.9610	0.8937
d4_dropout Δ% vs BL	−8.2%	−5.8%	−0.5%	−2.2%	−2.4%	−3.9%

3. **Scale-dependent benefit.** Our experiments use Moirai 2.0 Small (11.4M parameters). Preliminary results on the Base model (46M parameters) are inconsistent across seeds (Appendix J), suggesting the benefit may be specific to capacity-constrained models where the cross-patch information bottleneck is more binding.

4. **Fixed coefficients.** Fixed Chebyshev coefficients slightly outperform learned FIR filters (§5), suggesting the polynomial structure is already near-optimal for this setting. However, this means the approach has limited room for task-specific adaptation.

5. **Training data gap.** Our retraining uses 27B observations (public LOTSA) versus 295B in the official Moirai 2.0 release. While our controlled comparisons are valid (all conditions use the same data), the absolute performance gap to the official model suggests our findings should be verified at full data scale.

9 Discussion and Conclusion

In this paper, we adapt the ideas presented in Marsden and Hazan [24] to zero-shot patch-based time-series transformers. Concatenating the preconditioned signal as an auxiliary channel avoids the error accumulation of target-side preconditioning at multi-step horizons, and adds only 0.11% parameters, while providing a significant 2.9% improvement on GIFT-Eval. At a 100K-step budget, the method remains substantially better than the matched baseline, with a larger relative gap and lower cross-seed variance. The improvement is also largest at longer horizon lengths.

More broadly, these results suggest that time-series foundation models can still benefit substantially from simple, theory-guided input transformations. A fixed polynomial channel derived from USP theory improves zero-shot forecasting across benchmarks without significantly increasing parameter count or changing the transformer stack. This points to a practical path for making compact forecasters more accurate and parameter-efficient by injecting structure before tokenization, rather than relying only on larger models or heavier attention mechanisms.

The same principle may extend beyond time series. Pretrained language models and other sequence models could benefit from lightweight, fixed preconditioning channels that expose long-range structure before tokenization or early embedding layers, offering a complementary alternative to scaling model size alone.

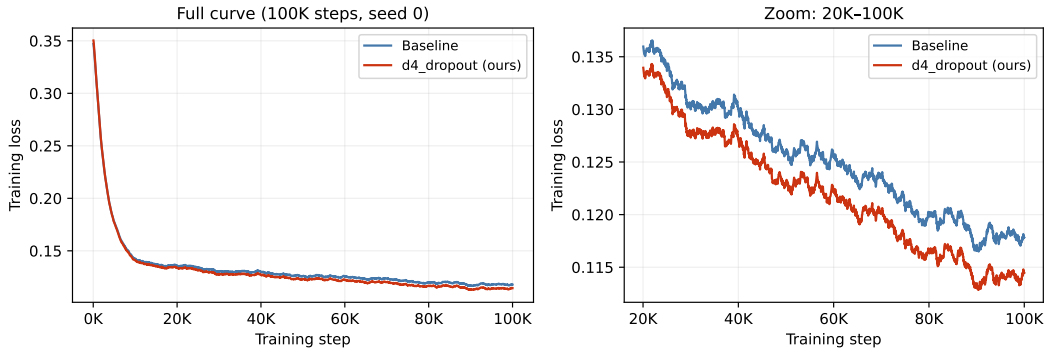

Figure 3: Training loss on LOTSA (seed 0, 100K steps, EMA-smoothed). Left: full curve. Right: zoom on steps 20K–100K. `d4_dropout` consistently tracks below Baseline throughout training, with the gap widening in the later stages.

Table 11: Horizon stratification of `d4` vs `d4_dropout` vs Baseline. 5-seed cross-config geomean MASE; $\Delta\%$ relative to BL within each bin.

Term	n	**BL**	`d4` (Δ)	`d4_dropout` (Δ)
short	55	0.840	**0.825** (-1.8%)	0.828 (-1.4%)
medium	21	0.859	0.830 (-3.4%)	**0.823** (-4.2%)
long	21	0.924	0.880 (-4.7%)	**0.874** (-5.3%)

References

[1] Naman Agarwal, Elad Hazan, Karan Singh, and Cyril Zhang. Spectral state space models. *arXiv preprint arXiv:2312.06837*, 2023.

[2] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.

[3] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. In *ICML*, 2024.

[4] Andreas Auer, Patrick Podest, Daniel Klotz, Sebastian Böck, Günter Klambauer, and Sepp Hochreiter. TiRex: Zero-shot forecasting across long and short horizons. In *ICML Workshop on Foundation Models for Structured Data (FMSD)*, 2025. URL https://openreview.net/forum?id=cWrbpOZGRa.

[5] George E. P. Box and Gwilym M. Jenkins. Time series analysis: Forecasting and control. *Holden-Day*, 1970.

[6] Yuxuan Chen, Qingsong Liu, and Yong Zhu. Multi-patch prediction: Adapting language models for time series representation learning. In *ICML*, 2024.

[7] Tri Dao, Naman Agarwal, Elad Hazan, et al. Flash STU: Fast spectral transform units. *arXiv preprint arXiv:2409.10489*, 2024.

[8] Abhimanyu Das, Weihao Kong, Andrew Leber, Rajat Mathews, and Ruhan Sen. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.

[9] Alexei Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[10] Vijay Ekambaram, Arindam Jati, Nam H. Nguyen, Pankaj Dayama, Chandra Reddy, Wesley Gifford, and Jayant Kalagnanam. TTMs: Fast multi-level tiny time mixers for improved zero-shot and few-shot forecasting of multivariate time series. In *NeurIPS*, 2024.

[11] Shanghua Gao, Teddy Koker, Owen Queen, Thomas Hartvigsen, Theodoros Tsiligkaridis, and Marinka Zitnik. Units: Building a unified time series model. In *ICML*, 2024.

[12] Azul Garza and Max Mergenthaler-Canseco. Timegpt-1. *arXiv preprint arXiv:2310.03589*, 2023.

[13] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024.

[14] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[15] Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022.

[16] Lu Han, Han-Jia Ye, and De-Chuan Zhan. The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting. *arXiv preprint arXiv:2304.05206*, 2024.

[17] Elad Hazan, Karan Singh, and Cyril Zhang. Learning linear dynamical systems via spectral filtering. In *NeurIPS*, 2017.

[18] Elad Hazan, Holden Lee, Karan Singh, Cyril Zhang, and Yi Zhang. Spectral filtering for general linear dynamical systems. In *NeurIPS*, 2018.

[19] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *ICLR*, 2022.

[20] Yuxuan Liang, Haomin Wen, Yuqi Nie, Yushan Jiang, Ming Jin, Dongjin Song, Shirui Pan, and Qingsong Wen. Foundation models for time series analysis: A tutorial and survey. *arXiv preprint arXiv:2403.14735*, 2024.

[21] Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Hanshu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698*, 2025.

[22] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *ICLR*, 2024.

[23] Yong Liu, Haoran Zhang, Changyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. *arXiv preprint arXiv:2402.02368*, 2024.

[24] Annie Marsden and Elad Hazan. Universal sequence preconditioning. *arXiv preprint arXiv:2502.06545*, 2025.

[25] John C. Mason and David C. Handscomb. *Chebyshev Polynomials*. CRC Press, 2003.

[26] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*, 2023.

[27] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A survey on time series foundation models. *arXiv preprint arXiv:2405.02358*, 2024.

[28] Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Deep adaptive input normalization for time series forecasting. In *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

[29] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhatt, Marin Bilos, Hena Ghonia, Nadhir Vincent Hassen, Anderson Schneider, Sahil Thiele, and Johannes Gasthaus. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.

[30] Oleksandr Shchur, Caner Turkmen, Abdul Fatir Ansari, Lorenzo Stella, and Syama Sundar Rangapuram. FEV-Bench: A unified benchmark for few-shot time series forecasting evaluation. *arXiv preprint arXiv:2509.26468*, 2025.

[31] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

[32] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *ICML*, 2024.

[33] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Ning An, Defu Lian, Longbing Cao, and Zhendong Niu. Frequency-domain mlps are more effective learners in time series forecasting. In *NeurIPS*, 2024.

[34] Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *ICLR*, 2023.

[35] Yunkai Zhang, Qi Zeng, Yawen Zhang, Zhijie Xu, Ming Zheng, Chongyang Gao, Muyan Jiang, and Zeyu Zheng. Are time series foundation models ready for zero-shot forecasting? In *ICML Workshop on Foundation Models for Structured Data (FMSD)*, 2025. URL https://openreview.net/forum?id=kQDzqIkXLO.

[36] Tian Zhou, Ziqing Ma, Qingsong Wen, Liang Sun, Tao Yao, Wotao Yin, and Rong Jin. FiLM: Frequency improved legendre memory model for long-term time series forecasting. In *NeurIPS*, 2022.

[37] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *ICML*, 2022.

A Detailed Methodology

A.1 Training Protocol

All experiments use identical training infrastructure on NVIDIA H100/A100 GPUs. The complete configuration:

Parameter	Value
Architecture	Moirai 2.0 Small (6 layers, $d = 384$, $d_{\text{ff}} = 1024$, $P = 16$)
Total parameters	11.4M (baseline), 11.4M + 12K (`d4_dropout`)
Training data	Public LOTSA setup, 27B observations
Official reference	100K steps, 295B observations
Steps (10K runs)	100 epochs $\times$ 100 batches = 10,000 steps
Steps (100K runs)	1,000 epochs $\times$ 100 batches = 100,000 steps
Batch size	256
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$, wd $= 0.1$)
LR schedule	Cosine annealing, peak $\eta = 10^{-3}$, min $\eta = 0$
Warmup	1,000 steps (linear)
Precision	bf16 mixed precision
Anomaly filtering	z-score threshold = 8.0, variance ratio = 0.0 (disabled)
Seeds	0, 1, 2, 7, 42 (5 seeds for main results)

A.2 Evaluation Protocol

All evaluations use the GIFT-Eval benchmark [2] with 97 dataset$\times$horizon configurations:

- Context length: 4,000 tokens (maximum supported)
- Patch size: $P = 16$ (fixed in Moirai 2.0; the variable patch sizes $P \in \{8, 32\}$ apply to Moirai 1.x only)
- Batch size: 64
- Metric: Normalized MASE (each configuration's MASE divided by the seasonal naive baseline's MASE)
- Aggregation: geometric mean across 97 configurations
- Reported quantities: cross-seed mean and standard deviation, per-seed MASE, and per-configuration win counts (out of 97 per seed; $5 \times 97 = 485$ when pooled across seeds)

A.3 Preconditioning Channel Implementation

The monic Chebyshev polynomial $\tilde{T}_d(z) = z^d + c_1 z^{d-1} + \cdots + c_d$ is expanded in the monomial basis to obtain the coefficients used by the preconditioning filter. The preconditioning residual for time step t is:

$$r_t = \left(x_t + \sum_{k=1}^{d} c_k \, x_{t-ks} \right) - x_t$$

This is computed over the full (normalized, zero-mean unit-variance) time series before patching. Unobserved time steps are masked to zero. During training with channel dropout p, each patch's preconditioning channel independently has probability p of being replaced with zeros. At inference, the full preconditioning channel is always provided.

The preconditioning channel is concatenated with the target and mask channels before the input projection:

$$\mathbf{e}_i = \text{ResBlock}_{3P \to d}([\mathbf{p}_i; \mathbf{m}_i; \mathbf{r}_i])$$

B Training Loss Analysis

Table 12: Final training loss across 5 seeds at 10K steps ($\eta = 10^{-3}$). Training loss differences are small ($\sim$1%), while eval MASE differences are much larger ($\sim$3%).

Method	**s0**	**s1**	**s2**	**s7**	**s42**	**Mean**	**Std**	**Eval MASE**
Baseline	.1366	.1365	.1358	.1366	.1365	.1364	.0003	0.8617
`d4_dropout` (ours)	.1354	.1350	.1350	.1353	.1343	**.1350**	.0004	**0.8368**
Zero	.1378	.1369	.1363	.1366	.1353	.1366	.0008	0.8578
Duplicate	.1376	.1366	.1370	.1365	.1355	.1366	.0007	0.8650
`d4_dropout` Δ% vs BL						-1.0%		-2.9%

Table 13: Final training loss across seeds at 100K steps ($\eta = 10^{-3}$). `d4_dropout` maintains lower training loss on every seed.

Method	**s0**	**s1**	**s2**	**s7**	**s42**	**Mean**	**Std**	**Eval MASE**
Baseline	.1185	.1185	.1190	.1194	.1179	.1187	.0005	0.8784
`d4_dropout` (ours)	.1144	.1170	.1155	.1172	.1155	**.1159**	.0010	**0.8443**
Zero	.1186	.1188	.1184	.1188	.1187	.1187	.0002	0.8937
Duplicate	.1192	.1193	.1185	.1197	.1182	.1190	.0006	0.8860
`d4_dropout` Δ% vs BL						-2.4%		-3.9%

C Per-Seed Results (All Conditions)

Table 14: Per-seed results. Both `d4` and `d4_dropout` improve over Baseline and controls on every seed.

Seed	**Baseline**	**`d4` (Ours)**	**`d4_dropout` (Ours)**	**Zero**	**Duplicate**
0	0.8666	0.8327	**0.8234**	0.8586	0.8777
1	0.8503	**0.8435**	0.8465	0.8738	0.8546
2	0.8630	**0.8283**	0.8391	0.8425	0.8662
7	0.8676	0.8422	**0.8320**	0.8573	0.8602
42	0.8614	0.8425	**0.8430**	0.8569	0.8661
Mean	0.8617	0.8379	**0.8368**	0.8578	0.8650

D Matched Learning Rate

Table 15: Learning rate $\times$ method interaction (5-seed mean MASE).

	$\eta = 10^{-3}$	$\eta = 2 \times 10^{-3}$	LR sensitivity
Baseline	0.8617	0.8422	2.27%
`d4_dropout` (Ours)	0.8368	0.8361	0.09%
`d4_dropout` vs BL	-2.89% ($p < 10^{-15}$)	-0.72% ($p = 3.8 \times 10^{-4}$)	

E Domain and Frequency Analysis

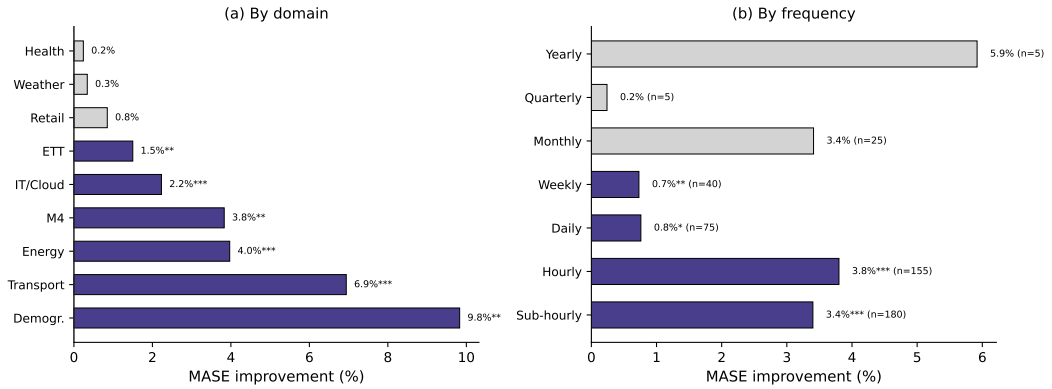

Figure 4: Domain and frequency analysis (`d4_dropout` vs. Baseline, 5 seeds pooled). Green bars: significant at $p < 0.05$. Gray: not significant.

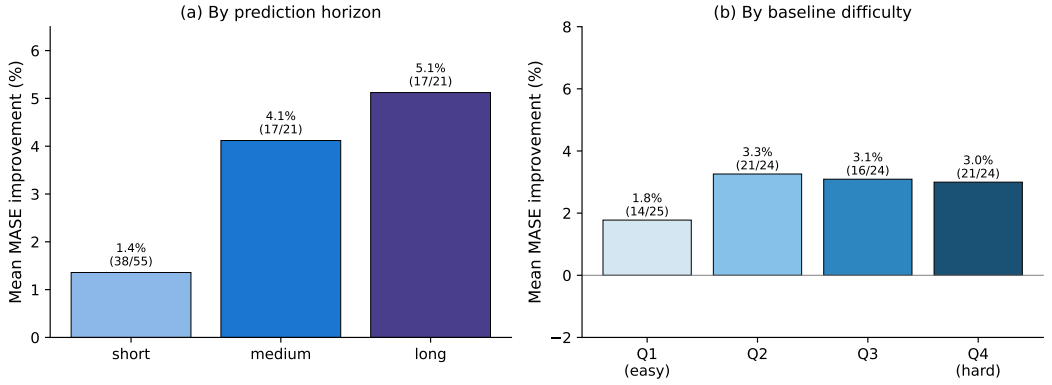

Figure 5: (a) Improvement by prediction horizon (short/medium/long). (b) Improvement by baseline difficulty quartile. Harder configurations benefit more.

F Horizon Analysis: `d4` vs `d4_dropout`

Table 16: `d4` vs `d4_dropout` vs Baseline by GIFT-Eval term. Bold marks the better of `d4`/`d4_dropout` in each bin.

Bin	n	**BL**	**`d4`**	**`d4_dropout`**	**`d4` $\Delta\%$**	**`d4_dropout` $\Delta\%$**
short	55	0.8403	**0.8252**	0.8282	-1.79%	-1.44%
medium	21	0.8592	0.8301	**0.8231**	-3.39%	-4.20%
long	21	0.9237	0.8802	**0.8744**	-4.71%	-5.34%
ALL	97	0.8618	0.8379	**0.8368**	-2.77%	-2.90%

Table 17: `d4` vs `d4_dropout` vs Baseline by raw prediction-length bin.

Bin	n	**BL**	**d4**	**d4_dropout**	**d4** $\Delta\%$	**d4_dropout** $\Delta\%$
PL ≤ 32	30	1.0237	**1.0074**	1.0098	-1.59%	-1.36%
$32 <$ PL ≤ 64	25	0.6631	**0.6496**	0.6529	-2.04%	-1.55%
$480 \leq$ PL ≤ 600	21	0.8592	0.8301	**0.8231**	-3.39%	-4.20%
PL ≥ 720	21	0.9237	0.8802	**0.8744**	-4.71%	-5.34%

G FEV-Bench Evaluation

To verify that our findings generalize beyond GIFT-Eval, we evaluate on FEV-Bench (100 tasks), an independent time series forecasting benchmark, and reproduce the polynomial-degree sweep over $d \in \{2, \ldots, 7\}$.

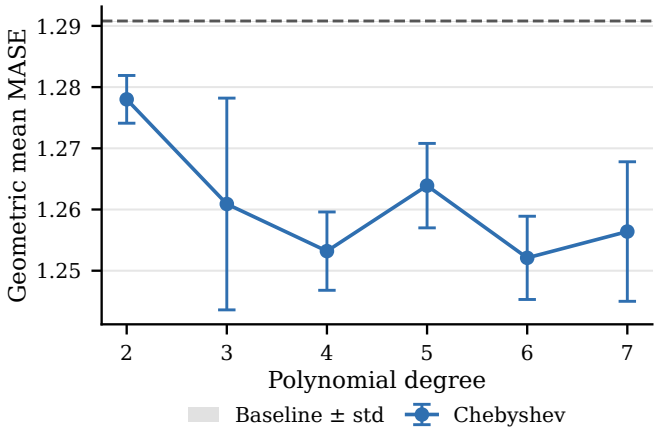

Figure 6: FEV-Bench polynomial-degree sweep (5 seeds per d, stride s=16, 10% channel dropout). Errorbars are cross-seed std; dashed line is the 5-seed baseline mean. Every tested $d \geq 2$ beats baseline, mirroring the GIFT-Eval pattern (Figure 2).

Table 18: FEV-Bench polynomial-degree sweep, 5 seeds per degree. $\Delta\%$ relative to the 5-seed baseline mean (1.282).

Degree	**MASE (mean $\pm$ std)**	**$\Delta\%$ vs BL**	**SQL (mean $\pm$ std)**
2	1.2780 ± 0.0039	-0.99%	1.0425 ± 0.0032
3	1.2609 ± 0.0173	-2.32%	1.0288 ± 0.0125
4	1.2532 ± 0.0064	-2.92%	1.0236 ± 0.0053
5	1.2639 ± 0.0069	-2.08%	1.0309 ± 0.0063
6	1.2521 ± 0.0068	-3.00%	1.0236 ± 0.0060
7	1.2564 ± 0.0114	-2.66%	1.0263 ± 0.0094

H Per-Configuration Analysis

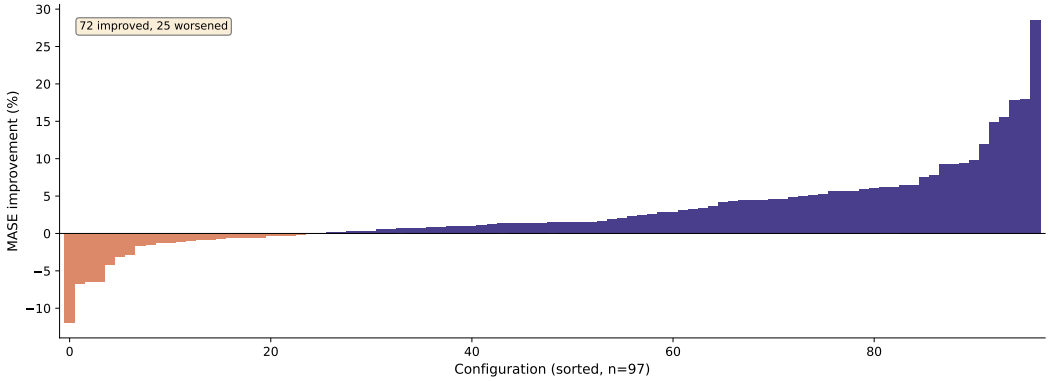

Figure 7: Per-configuration improvement (5-seed average), sorted. Green: `d4_dropout` wins. Red: baseline wins. 72/97 configurations improve.

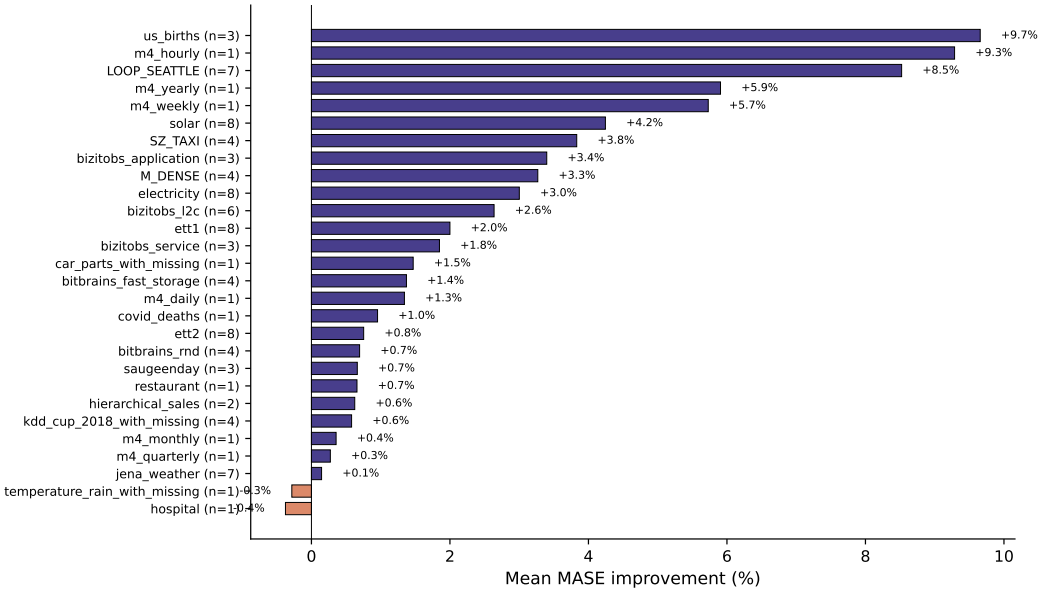

Figure 8: Improvement by dataset (5-seed average). Transport and energy datasets benefit most; weather is neutral.

I Context Length

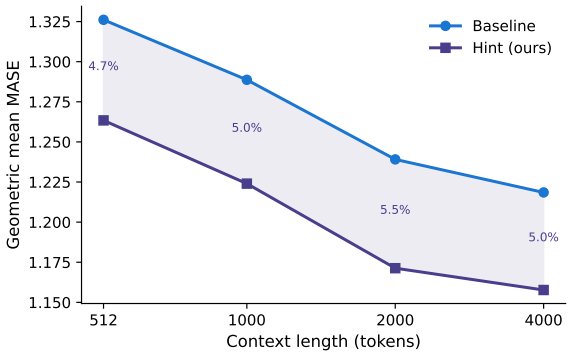

Figure 9: The improvement from polynomial preconditioning is approximately constant ($\sim$5%) across context lengths 512–4,000. `d4_dropout` at context 2,000 (0.8331) outperforms the baseline at 4,000 (0.8666).

Context	BL	`d4_dropout`	Δ%	Wins/97
512	0.9432	0.8986	-4.73%	62
1,000	0.9166	0.8706	-5.02%	65
2,000	0.8813	0.8331	-5.47%	71
4,000	0.8666	0.8234	-4.99%	66

J Base Model Scale

Table 19: Base model (46M parameters, 10K steps). Results are inconsistent across seeds.

Seed	BL (46M)	`d4_dropout` (46M)	Δ%	Wins
0	0.8504	0.8520	-0.19%	52/95
1	0.8473	0.8509	-0.42%	60/95
42	0.8570	0.8417	$+1.80\%$	59/93

K Complete Ablation Catalog

K.1 Learned Coefficients (5 seeds)

Table 20: Learned coefficient ablation (5 seeds, 10K steps). Per-seed learned coefficients and MASE.

Seed	Learned (c_1, c_2, c_3, c_4)	MASE	Δ%
Chebyshev init (mean MASE = 0.844)			
0	$[-0.026, -1.032, -0.123, +0.007]$	0.833	-3.9%
1	$[+0.010, -1.034, -0.134, -0.000]$	0.843	-0.8%
2	$[-0.016, -1.027, -0.152, -0.011]$	0.856	-4.0%
7	$[+0.002, -1.028, -0.141, +0.020]$	0.842	-2.9%
42	$[-0.032, -1.030, -0.148, -0.015]$	0.846	-2.2%
Zero init (mean MASE = 0.839)			
0	$[+0.308, +0.177, +0.057, +0.045]$	0.836	-3.6%
1	$[-0.286, -0.188, -0.054, -0.044]$	0.847	-0.4%
2	$[-0.323, -0.190, -0.058, -0.066]$	0.830	-4.0%
7	$[-0.303, -0.168, -0.057, -0.048]$	0.839	-3.3%
42	$[-0.331, -0.163, -0.028, -0.057]$	0.842	-2.2%
Fixed Chebyshev (mean MASE = 0.837, not learned)			

K.2 Zero-Hint Dependence vs Dropout Rate (seed 0)

Train dropout	Normal MASE	Zero-hint MASE	Degradation
0%	0.8534	1.1594	$+35.9\%$
2%	0.8388	0.8996	$+7.2\%$
10% (`d4_dropout`)	0.8234	0.9003	$+9.3\%$
30%	0.8559	0.8769	$+2.5\%$

K.3 Computational Overhead

Measured across 9 paired runs (`d4_dropout` and BL on the same GPU node):

	Baseline	`d4_dropout`
Training throughput	2.18 it/s	2.14 it/s (-1.6%)
Extra parameters	0	12,288 ($+0.11\%$)
Peak GPU memory	identical	identical

L Matched-Official Protocol (100K Steps, 10K Warmup)

The 100K results in the main paper use a 1,000-step warmup, while the official Moirai 2.0 training protocol [21] uses 10,000 steps of warmup. To control for this protocol difference, we retrain both baseline and `d4_dropout` at 100K steps with the exact official hyperparameters.

Table 21: Matched-official 100K results (10K warmup, cosine annealing to 0, otherwise identical to official protocol).

Method	s0	s2	s7	Mean	vs BL
Baseline (1K warmup)	0.909	0.863	0.864	0.878	—
Baseline (10K warmup)	0.882	0.880	0.917	0.893	—
`d4_dropout` (1K warmup)	0.835	0.859	0.845	0.844	-3.9%
`d4_dropout` (10K warmup)	0.892	0.846	0.880	**0.873**	$\mathbf{-2.3\%}$

With 10K warmup (matching the official protocol), `d4_dropout` still outperforms the baseline by 2.3% on average across 3 seeds (compared to 3.9% with 1K warmup). The smaller advantage reflects that the longer warmup partially stabilizes the baseline—at seed 0, BL improves from 0.909 (1K warmup, which shows the late-training collapse) to 0.882 (10K warmup). The benefit is seed-dependent: `d4_dropout` wins at seed 2 ($+3.8\%$) and seed 7 ($+4.0\%$), but slightly loses at seed 0 (-1.1%). This suggests that part of `d4_dropout`'s 100K advantage at 1K warmup is robustness to schedule-induced instability, but a genuine $\sim$2% inductive-bias effect persists under the matched-official protocol.